

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN CÁC HỆ THỐNG PHÂN TÁN

Giảng viên: Kim Ngọc Bách

Nhóm bài tập: 11

Lớp : 0 2

Sinh viên : Lê Tiến Đạt - B22DCCN188

Nguyễn Đức Phúc- B22DCCN631

Nguyễn Ngọc Tân-B22DCCN715

Hà Nội - 2026

MỤC LỤC

LỜI MỞ ĐẦU	1
DANH MỤC BẢNG BIỂU, HÌNH ẢNH.....	2
DANH MỤC TỪ VIẾT TẮT	3
CHƯƠNG 1. MÔ TẢ BÀI TOÁN	4
1.1. Giới thiệu bài toán	4
1.2. Nội dung bài toán	4
1.3. Yêu cầu chức năng	5
1.4. Yêu cầu phi chức năng	5
1.5. Phạm vi hệ thống	6
1.6. Tổng kết chương.....	6
CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG	7
2.1. Định hướng kiến trúc.....	7
2.2. Thành phần kiến trúc	7
2.3. Mô hình giao tiếp	8
2.4. Sơ đồ kiến trúc logic.....	8
2.5. Vai trò của API Gateway	9
2.6. Cơ sở dữ liệu riêng cho từng service.....	9
CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE	11
3.1. API Gateway	11
3.2. User Service	11
3.3. Merchant Service.....	12
3.4. Order Service.....	12
3.5. Delivery Service	12

3.6. Notification Service.....	13
3.7. Frontend và Live Admin Dashboard.....	13
CHƯƠNG 4. THIẾT KẾ RESTFUL API.....	14
4.1. Nguyên tắc thiết kế API	14
4.2. Xác thực và phân quyền	14
4.3. Nhóm API chính.....	14
4.4. Luồng API tiêu biểu	15
4.5. Chuẩn phản hồi và xử lý lỗi	15
CHƯƠNG 5. LUỒNG MESSAGE QUEUE	16
5.1. Vai trò của RabbitMQ	16
5.2. Các sự kiện chính trong hệ thống.....	16
5.3. Luồng tạo đơn và phát sự kiện	16
5.4. Luồng cập nhật trạng thái giao hàng	16
5.5. Xử lý lỗi và tính chịu lỗi	17
5.6. Ý nghĩa của Message Queue trong đề tài.....	17
CHƯƠNG 6. THỬ NGHIỆM VÀ ĐÁNH GIÁ	18
6.1. Mục tiêu thử nghiệm	18
6.2. Môi trường thử nghiệm	18
6.3. Kịch bản kiểm thử	18
6.4. Kết quả thử nghiệm	19
6.5. Đánh giá ưu điểm	20
6.6. Hạn chế và hướng phát triển	21
CHƯƠNG 7. HƯỚNG DẪN CÀI ĐẶT.....	22
7.1. Yêu cầu môi trường.....	22
7.2. Tải mã nguồn.....	22
7.3. Khởi chạy hạ tầng Docker.....	22
7.4. Cài đặt dependencies	22
7.5. Khởi chạy toàn bộ hệ thống	23
7.6. Kiểm thử nhanh hệ thống	23
7.7. Chạy toàn bộ bằng Docker Compose	23

7.8. Một số lỗi thường gặp	24
KẾT LUẬN	25
TÀI LIỆU THAM KHẢO	26

LỜI MỞ ĐẦU

Trong bối cảnh các hệ thống phần mềm ngày càng có quy mô lớn và yêu cầu xử lý đồng thời nhiều nghiệp vụ khác nhau, kiến trúc Microservices trở thành một hướng tiếp cận quan trọng trong phát triển phần mềm hiện đại. Việc phân rã hệ thống thành các dịch vụ độc lập không chỉ giúp tăng khả năng mở rộng mà còn hỗ trợ quá trình bảo trì, triển khai và kiểm thử một cách linh hoạt hơn.

Báo cáo này trình bày quá trình phân tích, thiết kế và xây dựng dự án BiteSwift - một hệ thống đặt đồ ăn trực tuyến và điều phối giao hàng được phát triển theo kiến trúc Microservices. Hệ thống sử dụng RESTful API cho các tương tác đồng bộ, RabbitMQ cho giao tiếp bất đồng bộ và WebSocket để cập nhật thông tin thời gian thực trên giao diện giám sát.

Thông qua đề tài, nhóm vận dụng các kiến thức của môn Các hệ thống phân tán vào một bài toán thực tế, bao gồm tổ chức service độc lập, thiết kế giao tiếp liên dịch vụ, xử lý sự kiện, tách cơ sở dữ liệu theo miền nghiệp vụ và xây dựng dashboard quan sát trạng thái hệ thống. Báo cáo được trình bày theo các nội dung chính: mô tả bài toán, kiến trúc hệ thống, mô tả microservice, thiết kế API, luồng message queue, thử nghiệm đánh giá và hướng dẫn cài đặt.

Nhóm xin chân thành cảm ơn giảng viên hướng dẫn đã cung cấp kiến thức nền tảng và định hướng thực hiện đề tài. Những góp ý trong quá trình học tập là cơ sở quan trọng để nhóm hoàn thiện sản phẩm và báo cáo này.

DANH MỤC BẢNG BIỂU, HÌNH ẢNH

1. Danh mục bảng biểu

STT	Danh mục bảng biểu	Trang
1	Bảng 1.1. Phạm vi các thành phần chính của hệ thống BiteSwift	6
2	Bảng 2.1. Thành phần kiến trúc và công nghệ sử dụng	7
3	Bảng 2.2. So sánh giao tiếp trong hệ thống	8
4	Bảng 3.1. Tóm tắt vai trò các microservice	11
5	Bảng 4.1. Nhóm API chính của hệ thống	14
6	Bảng 5.1. Các sự kiện chính trên RabbitMQ	16
7	Bảng 6.1. Môi trường thử nghiệm	18
8	Bảng 6.2. Kịch bản kiểm thử hệ thống	18
9	Bảng 7.1. Yêu cầu môi trường triển khai	22
10	Bảng 7.2. Cổng truy cập và thành phần tương ứng	23

2. Danh mục hình ảnh

STT	Danh mục hình ảnh	Trang
1	Hình 1. Sơ đồ kiến trúc tổng thể của hệ thống BiteSwift	9
2	Hình 2. Kết quả đặt món và cập nhật trạng thái giao hàng theo thời gian thực	19
3	Hình 3. Live Dashboard hiển thị luồng hệ thống, log tập trung và trạng thái hoạt động của các microservice	20

DANH MỤC TỪ VIẾT TẮT

Thuật ngữ	Nghĩa tiếng Anh	Nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng, dùng để các hệ thống trao đổi dữ liệu.
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản khi làm việc với dữ liệu.
DB	Database	Cơ sở dữ liệu dùng để lưu trữ thông tin của hệ thống.
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải dữ liệu trên web.
JWT	JSON Web Token	Chuỗi token dùng để xác thực và định danh người dùng.
MQ	Message Queue	Hàng đợi thông điệp dùng cho giao tiếp bất đồng bộ.
REST	Representational State Transfer	Phong cách thiết kế API dựa trên tài nguyên và phương thức HTTP.
SPA	Single Page Application	Ứng dụng web một trang, cập nhật nội dung mà không tải lại toàn bộ trang.
WebSocket	WebSocket	Giao thức hỗ trợ giao tiếp hai chiều theo thời gian thực giữa client và server.
AMQP	Advanced Message Queuing Protocol	Giao thức truyền thông điệp được RabbitMQ hỗ trợ.

CHƯƠNG 1. MÔ TẢ BÀI TOÁN

1.1. Giới thiệu bài toán

BiteSwift là hệ thống đặt đồ ăn trực tuyến được xây dựng nhằm mô phỏng quy trình vận hành của một nền tảng giao đồ ăn hiện đại. Hệ thống cho phép người dùng đăng nhập, tra cứu nhà hàng, lựa chọn món ăn, tạo đơn hàng và theo dõi trạng thái giao hàng thông qua giao diện web. Bên cạnh luồng thao tác của khách hàng, hệ thống còn cung cấp dashboard giám sát nhằm quan sát trạng thái service, log xử lý và luồng truyền message trong hệ thống.

Bài toán đặt đồ ăn trực tuyến có đặc điểm là bao gồm nhiều nghiệp vụ liên quan nhưng có thể tách biệt về mặt trách nhiệm. Quản lý người dùng, quản lý nhà hàng, xử lý đơn hàng, điều phối giao hàng và gửi thông báo đều là những miền nghiệp vụ tương đối độc lập. Khi triển khai theo kiến trúc nguyên khối, toàn bộ các chức năng này cùng nằm trong một ứng dụng, dẫn đến khó mở rộng và khó bảo trì khi hệ thống phát triển.

Mục tiêu của đề tài là xây dựng một ứng dụng theo kiến trúc Microservices, trong đó mỗi service chịu trách nhiệm một miền nghiệp vụ riêng và giao tiếp thông qua các cơ chế phù hợp. RESTful API được sử dụng cho các thao tác cần phản hồi trực tiếp, trong khi RabbitMQ Message Queue được sử dụng cho các nghiệp vụ bất đồng bộ như phát sinh sự kiện đơn hàng, điều phối giao hàng và gửi thông báo. Cách tổ chức này giúp hệ thống thể hiện rõ các đặc trưng của hệ thống phân tán: tách rời thành phần, giao tiếp liên dịch vụ, xử lý sự kiện và khả năng quan sát.

1.2. Nội dung bài toán

Hệ thống cần hỗ trợ luồng nghiệp vụ cơ bản của một nền tảng giao đồ ăn. Người dùng truy cập giao diện, đăng nhập bằng tài khoản mẫu, lựa chọn nhà hàng và món ăn, thêm món vào giỏ hàng, nhập địa chỉ giao hàng và xác nhận đặt món. Sau khi đơn hàng được tạo, hệ thống không xử lý toàn bộ các bước phía sau trong cùng một tiến trình mà phát sinh sự kiện để các service khác tiếp tục xử lý.

Order Service chịu trách nhiệm tiếp nhận yêu cầu đặt hàng, kiểm tra thông tin món ăn bằng cách gọi đồng bộ tới Merchant Service, ghi nhận đơn hàng vào cơ sở dữ liệu riêng và gửi sự kiện `order.created` đến RabbitMQ. Delivery Service tiêu thụ sự kiện này, mô phỏng quá trình tìm tài xế, cập nhật trạng thái giao hàng qua từng giai đoạn và phát sinh sự kiện `delivery.status_changed`. Notification Service lắng nghe các sự kiện liên quan và phát cập nhật thời gian thực đến giao diện thông qua WebSocket.

Về chức năng giám sát, hệ thống cung cấp Live Admin Dashboard. Dashboard hiển

thị trạng thái hoạt động của các service, log tập trung và mô phỏng luồng message giữa các thành phần. Thành phần này có ý nghĩa quan trọng trong quá trình kiểm thử và trình bày đồ án, vì nó cho phép quan sát trực quan cách các service phối hợp trong kiến trúc phân tán.

1.3. Yêu cầu chức năng

Các chức năng của hệ thống được tổ chức theo các nhóm nghiệp vụ chính. Nhóm chức năng người dùng bao gồm đăng nhập, nhận JWT Token và sử dụng token trong các request cần xác thực. Nhóm chức năng nhà hàng bao gồm lưu trữ thông tin nhà hàng, thực đơn và món ăn. Nhóm chức năng đặt hàng bao gồm tạo đơn, kiểm tra thông tin món ăn, lưu đơn hàng và phát sinh sự kiện đơn hàng mới.

Nhóm chức năng giao hàng do Delivery Service đảm nhiệm. Service này nhận sự kiện đơn hàng mới từ RabbitMQ, tìm tài xế phù hợp theo dữ liệu người dùng, tạo tiến trình giao hàng và cập nhật trạng thái đơn hàng. Các trạng thái được mô phỏng tuần tự nhằm thể hiện rõ quá trình xử lý nền trong hệ thống. Nhóm chức năng thông báo do Notification Service đảm nhiệm, bao gồm tiếp nhận sự kiện, gửi cập nhật thời gian thực và gom log tập trung từ các service.

Ngoài các chức năng nghiệp vụ, hệ thống còn cung cấp chức năng giám sát. Dashboard giúp hiển thị trạng thái health check của các service, luồng message qua RabbitMQ và log phát sinh trong quá trình xử lý. Đây là một yêu cầu quan trọng đối với đề tài hệ thống phân tán vì khả năng quan sát giúp đánh giá mức độ hoạt động độc lập và phối hợp giữa các service.

1.4. Yêu cầu phi chức năng

Hệ thống cần có khả năng mở rộng theo từng service. Do các thành phần được tách biệt theo miền nghiệp vụ, việc phát triển, kiểm thử và triển khai có thể thực hiện độc lập hơn so với kiến trúc nguyên khối. Khi nhu cầu xử lý tăng, các service như Order Service hoặc Notification Service có thể được ưu tiên mở rộng mà không cần thay đổi toàn bộ ứng dụng.

Tính ổn định được đảm bảo thông qua cơ chế giao tiếp bất đồng bộ. RabbitMQ đóng vai trò là trung gian tiếp nhận và phân phối sự kiện, giúp giảm sự phụ thuộc trực tiếp giữa các service. Trong trường hợp một service xử lý nền tạm thời chưa sẵn sàng, message vẫn có thể được lưu trong hàng đợi để xử lý sau.

Hệ thống sử dụng JWT nhằm cung cấp cơ chế xác thực cơ bản. API Gateway là điểm vào chung, chịu trách nhiệm tiếp nhận request từ client và định tuyến đến service phù hợp. Mô hình này giúp kiểm soát truy cập tốt hơn và hạn chế việc client gọi trực tiếp tới

các service nội bộ. Bên cạnh đó, dashboard và log tập trung hỗ trợ khả năng quan sát trong quá trình vận hành, kiểm thử và đánh giá.

1.5. Phạm vi hệ thống

Bảng 1.1. Phạm vi các thành phần chính của hệ thống BiteSwift

Thành phần	Vai trò
Frontend	Giao diện người dùng và Live Admin Dashboard
API Gateway	Tiếp nhận request, xác thực JWT và định tuyến request
User Service	Quản lý tài khoản, vai trò người dùng và phát hành JWT
Merchant Service	Quản lý nhà hàng, thực đơn và món ăn
Order Service	Xử lý giỏ hàng, tạo đơn và phát sự kiện order.created
Delivery Service	Tiêu thụ sự kiện đơn hàng, mô phỏng giao hàng và cập nhật trạng thái
Notification Service	Lắng nghe sự kiện, gom log và phát WebSocket
RabbitMQ	Trung gian message queue cho giao tiếp bất đồng bộ
PostgreSQL / MongoDB	Lưu trữ dữ liệu riêng cho các service

Trong phạm vi đồ án, hệ thống tập trung vào việc mô phỏng quy trình đặt món và điều phối giao hàng. Các chức năng nâng cao như thanh toán trực tuyến, tích hợp bản đồ GPS, định vị tài xế thực tế, mã giảm giá, đánh giá món ăn và triển khai cloud production chưa được thực hiện. Những nội dung này có thể được xem là hướng phát triển trong tương lai.

1.6. Tổng kết chương

Chương 1 đã trình bày bối cảnh, mục tiêu, nội dung bài toán và phạm vi của hệ thống BiteSwift. Hệ thống được xây dựng nhằm mô phỏng một nền tảng đặt đồ ăn trực tuyến theo kiến trúc Microservices, trong đó các service phối hợp với nhau thông qua RESTful API và RabbitMQ. Các yêu cầu chức năng và phi chức năng được xác định trong chương này là cơ sở cho việc thiết kế kiến trúc, mô tả service, thiết kế API và xây dựng luồng message ở các chương tiếp theo.

CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG

2.1. Định hướng kiến trúc

BiteSwift được thiết kế theo kiến trúc Microservices. Thay vì triển khai toàn bộ nghiệp vụ trong một ứng dụng duy nhất, hệ thống được phân rã thành các service nhỏ hơn, mỗi service đảm nhiệm một miền nghiệp vụ cụ thể. Việc phân rã này giúp giảm sự phụ thuộc giữa các thành phần và tăng khả năng phát triển độc lập.

Trong kiến trúc tổng thể, Frontend chỉ giao tiếp trực tiếp với API Gateway và Notification Service. API Gateway đóng vai trò là điểm vào chính cho các request REST. Các backend service không được xem như những thành phần giao tiếp trực tiếp với người dùng cuối, mà hoạt động trong lớp dịch vụ bên trong. RabbitMQ đóng vai trò hạ tầng trung gian cho các sự kiện phát sinh trong hệ thống.

Hệ thống áp dụng nguyên tắc mỗi service quản lý dữ liệu riêng. User Service, Order Service và Delivery Service sử dụng PostgreSQL cho dữ liệu quan hệ, trong khi Merchant Service sử dụng MongoDB phù hợp với dữ liệu nhà hàng và thực đơn có cấu trúc linh hoạt. Cách tiếp cận này hạn chế việc truy vấn trực tiếp dữ liệu của service khác và khuyến khích giao tiếp thông qua API hoặc sự kiện.

2.2. Thành phần kiến trúc

Bảng 2.1. Thành phần kiến trúc và công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Frontend	React, Vite, CSS	Cung cấp giao diện đặt món và dashboard giám sát
API Gateway	Node.js, Express	Định tuyến request, kiểm tra token, tập trung điểm vào
User Service	Node.js, PostgreSQL, JWT, bcrypt	Quản lý tài khoản, vai trò và xác thực
Merchant Service	Node.js, MongoDB	Quản lý nhà hàng, thực đơn và món ăn
Order Service	Node.js, PostgreSQL, AMQP	Tạo đơn hàng, kiểm tra món ăn, phát sự kiện
Delivery Service	Node.js, PostgreSQL, AMQP	Xử lý giao hàng và cập nhật trạng

		thái
Notification Service	Node.js, Socket.io, AMQP	Gom log, lắng nghe sự kiện và phát WebSocket
RabbitMQ	AMQP Broker	Truyền thông điệp bất đồng bộ giữa các service
Docker Compose	Container orchestration local	Khởi chạy hạ tầng dữ liệu và message broker

2.3. Mô hình giao tiếp

BiteSwift kết hợp hai mô hình giao tiếp: đồng bộ và bất đồng bộ. Giao tiếp đồng bộ được sử dụng khi service cần kết quả ngay để hoàn tất nghiệp vụ hiện tại. Ví dụ, khi tạo đơn hàng, Order Service gọi Merchant Service để kiểm tra giá và tính khả dụng của món ăn. Nếu thông tin món ăn không hợp lệ, đơn hàng không được tạo.

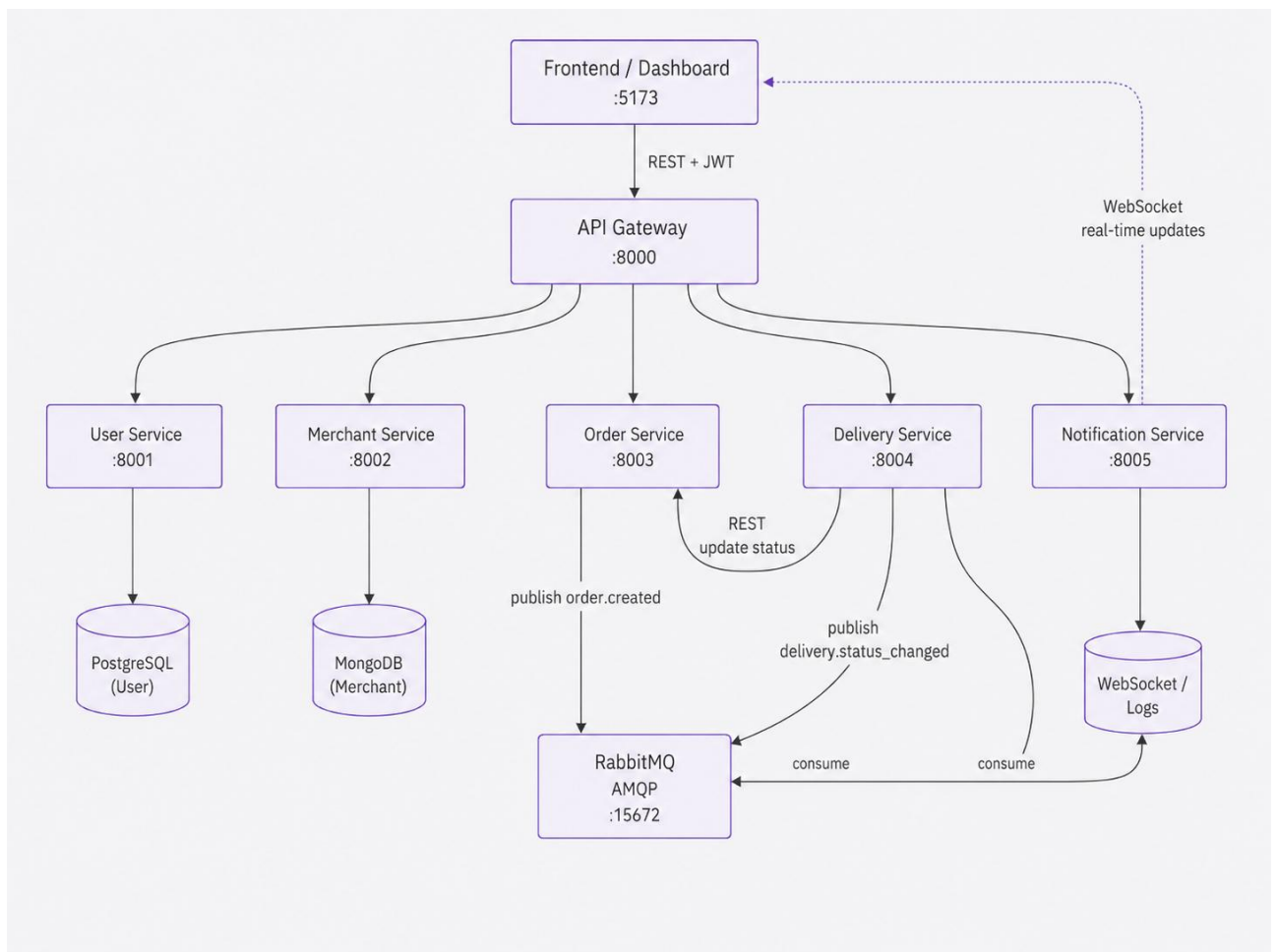
Giao tiếp bất đồng bộ được sử dụng cho các công việc có thể xử lý nền. Sau khi đơn hàng được ghi nhận, Order Service không cần chờ Delivery Service hoàn tất toàn bộ quá trình giao hàng. Thay vào đó, Order Service phát sự kiện `order.created` vào RabbitMQ. Delivery Service và Notification Service có thể tiêu thụ sự kiện để thực hiện phần việc riêng. Cách tổ chức này làm giảm độ phụ thuộc trực tiếp giữa các service và tăng khả năng chịu lỗi của hệ thống.

Bảng 2.2. So sánh giao tiếp trong hệ thống

Cơ chế	Khi sử dụng	Ví dụ
RESTful API	Khi cần phản hồi trực tiếp	Đăng nhập, xem nhà hàng, kiểm tra món ăn, cập nhật trạng thái đơn
Message Queue	Khi xử lý nền hoặc phát sự kiện	<code>order.created</code> , <code>delivery.status_changed</code>
WebSocket	Khi cần cập nhật thời gian thực	Thông báo trạng thái đơn hàng, log dashboard

2.4. Sơ đồ kiến trúc logic

Sơ đồ logic dưới đây mô tả cách các thành phần chính phối hợp với nhau trong hệ thống BiteSwift. Frontend gửi request đến API Gateway cho các thao tác REST. API Gateway định tuyến request đến các service nghiệp vụ. Các sự kiện phát sinh trong quá trình đặt hàng được.



Hình 1. Sơ đồ kiến trúc tổng thể của hệ thống BiteSwift

2.5. Vai trò của API Gateway

API Gateway là cổng vào chung của hệ thống. Thành phần này tiếp nhận các request từ client, kiểm tra thông tin xác thực và chuyển tiếp request tới service phù hợp. Việc sử dụng Gateway giúp đơn giản hóa giao tiếp từ phía frontend, vì client không cần biết chi tiết địa chỉ của từng service backend.

Gateway cũng góp phần tập trung xử lý các vấn đề chung như xác thực token, định tuyến, chuẩn hóa lỗi và log request. Trong các hệ thống Microservices, Gateway giúp giảm sự phức tạp ở phía client và tạo ra một lớp bảo vệ trước các service nội bộ.

2.6. Cơ sở dữ liệu riêng cho từng service

Một nguyên tắc quan trọng trong hệ thống là mỗi service chịu trách nhiệm sở hữu dữ liệu của chính nó. User Service không truy vấn trực tiếp cơ sở dữ liệu của Order Service, Order Service cũng không đọc trực tiếp dữ liệu của Merchant Service. Khi cần dữ liệu từ service khác, hệ thống sử dụng API hoặc event.

Cách tổ chức này giúp giảm sự phụ thuộc ở tầng dữ liệu và bảo vệ tính độc lập của từng service. Tuy nhiên, nó cũng đặt ra thách thức về nhất quán dữ liệu. Trong phạm vi dự án, nhất quán được xử lý theo hướng đơn giản: dữ liệu quan trọng được kiểm tra đồng bộ khi tạo đơn, còn các cập nhật trạng thái giao hàng được truyền thông qua event và REST.

CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE

Mỗi microservice trong BiteSwift được xây dựng như một đơn vị triển khai độc lập, có trách nhiệm nghiệp vụ rõ ràng và có thể chạy trên một cổng riêng. Việc phân chia service được thực hiện dựa trên miền nghiệp vụ của bài toán đặt đồ ăn: người dùng, nhà hàng, đơn hàng, giao hàng và thông báo.

Bảng 3.1. Tóm tắt vai trò các microservice

Service	Vai trò chính
API Gateway	Điểm vào chung, xác thực và định tuyến
User Service	Quản lý tài khoản khách hàng, nhà hàng, tài xế và token
Merchant Service	Quản lý thông tin nhà hàng và món ăn
Order Service	Xử lý giỏ hàng, tạo đơn và phát sự kiện
Delivery Service	Xử lý giao hàng, cập nhật trạng thái và phát sự kiện giao hàng
Notification Service	Gom log, nhận sự kiện và phát thông báo thời gian thực
Frontend	Cung cấp giao diện người dùng và dashboard giám sát

3.1. API Gateway

API Gateway là thành phần trung gian giữa frontend và các service backend. Thay vì frontend gọi trực tiếp đến từng service, toàn bộ request REST được gửi qua Gateway. Gateway xác định đường dẫn request, kiểm tra token nếu cần và chuyển tiếp đến service tương ứng.

Việc tập trung luồng truy cập qua Gateway giúp kiểm soát hệ thống tốt hơn. Các logic chung như log request, xử lý lỗi, kiểm tra JWT và định tuyến đều được đặt tại một điểm thống nhất. Điều này phù hợp với mô hình Microservices, nơi số lượng service tăng lên có thể khiến client trở nên phức tạp nếu phải tự quản lý địa chỉ từng service.

3.2. User Service

User Service quản lý thông tin tài khoản và vai trò người dùng trong hệ thống. Các loại người dùng được mô phỏng bao gồm khách hàng, nhà hàng và tài xế giao hàng. Service này chịu trách nhiệm đăng nhập, mã hóa mật khẩu bằng bcrypt và phát hành JWT Token sau khi xác thực thành công.

Dữ liệu của User Service được lưu trong PostgreSQL. Việc sử dụng cơ sở dữ liệu

quan hệ phù hợp với thông tin người dùng vì dữ liệu có cấu trúc rõ ràng, bao gồm tài khoản, mật khẩu đã mã hóa, vai trò và các thuộc tính liên quan. Delivery Service có thể gọi User Service để tìm tài xế trống trong quá trình xử lý giao hàng.

3.3. Merchant Service

Merchant Service đảm nhiệm miền nghiệp vụ nhà hàng và món ăn. Service này lưu trữ thông tin nhà hàng, danh sách món ăn và các thông tin mô tả liên quan. Dữ liệu được lưu trong MongoDB nhằm phù hợp với đặc điểm dữ liệu có cấu trúc linh hoạt, đặc biệt là thực đơn của nhà hàng.

Trong luồng đặt hàng, Merchant Service đóng vai trò là nguồn dữ liệu xác thực về món ăn. Khi người dùng tạo đơn, Order Service gọi đồng bộ tới Merchant Service để kiểm tra giá và trạng thái khả dụng của món. Cách xử lý này giúp đảm bảo đơn hàng được tạo dựa trên dữ liệu hiện tại của hệ thống.

3.4. Order Service

Order Service là service trung tâm của luồng đặt món. Service này tiếp nhận yêu cầu checkout từ người dùng, kiểm tra thông tin món ăn, tính toán giá trị đơn hàng, lưu đơn hàng vào cơ sở dữ liệu và phát sinh sự kiện `order.created`. Đây là service thể hiện rõ sự kết hợp giữa giao tiếp đồng bộ và bất đồng bộ trong hệ thống.

Ở bước kiểm tra món ăn, Order Service cần phản hồi chính xác cho người dùng nên sử dụng REST API để gọi Merchant Service. Sau khi đơn hàng được tạo thành công, các bước phía sau như phân công tài xế và gửi thông báo không cần chặn phản hồi của người dùng. Vì vậy, Order Service phát event vào RabbitMQ để các service khác xử lý.

Cơ sở dữ liệu của Order Service được tách riêng, lưu thông tin đơn hàng và trạng thái đơn. Delivery Service có thể cập nhật ngược trạng thái đơn hàng thông qua REST API khi quá trình giao hàng thay đổi. Điều này giúp Order Service tiếp tục là nơi quản lý trạng thái đơn hàng chính thức.

3.5. Delivery Service

Delivery Service là consumer của sự kiện `order.created`. Sau khi nhận sự kiện từ RabbitMQ, service này bắt đầu mô phỏng quy trình giao hàng. Quy trình gồm tìm tài xế trống, tạo bản ghi giao hàng, cập nhật trạng thái đơn và phát sinh các sự kiện trạng thái mới.

Trong hệ thống thực tế, việc điều phối tài xế có thể phụ thuộc vào vị trí, khoảng cách, thời gian rảnh và thuật toán tối ưu. Trong phạm vi đồ án, quá trình này được mô phỏng bằng việc tìm tài xế có sẵn và thay đổi trạng thái theo các khoảng thời gian. Cách mô phỏng đủ để minh họa bản chất xử lý nền và giao tiếp bất đồng bộ trong hệ thống phân tán.

3.6. Notification Service

Notification Service có hai nhiệm vụ chính. Nhiệm vụ thứ nhất là lắng nghe các sự kiện phát sinh từ RabbitMQ, bao gồm `order.created` và `delivery.status_changed`. Nhiệm vụ thứ hai là phát thông tin đến frontend hoặc dashboard thông qua Socket.io WebSocket.

Service này cũng đóng vai trò gom log tập trung. Các service khác có thể gửi log qua HTTP POST đến Notification Service. Sau đó, log được phát trực tiếp lên dashboard để người dùng quan sát quá trình hoạt động. Chức năng này làm tăng khả năng quan sát của hệ thống và hỗ trợ rất tốt cho quá trình demo.

3.7. Frontend và Live Admin Dashboard

Frontend được xây dựng dưới dạng ứng dụng web một trang. Giao diện gồm hai phần chính: Customer Storefront và Live Admin Dashboard. Customer Storefront phục vụ thao tác của khách hàng như đăng nhập, xem nhà hàng, chọn món và đặt hàng. Live Admin Dashboard phục vụ việc theo dõi trạng thái hệ thống.

Dashboard hiển thị health check của các service, log tập trung và topology mô phỏng luồng message qua RabbitMQ. Đây là điểm nổi bật của dự án vì không chỉ cung cấp chức năng người dùng mà còn giúp trực quan hóa cách hệ thống phân tán vận hành.

CHƯƠNG 4. THIẾT KẾ RESTFUL API

4.1. Nguyên tắc thiết kế API

RESTful API trong BiteSwift được thiết kế theo hướng tài nguyên. Các endpoint sử dụng phương thức HTTP phù hợp với mục đích thao tác dữ liệu, chẳng hạn GET để truy vấn, POST để tạo mới, PUT hoặc PATCH để cập nhật và DELETE để xóa dữ liệu khi cần. Cách đặt tên endpoint ưu tiên tính dễ hiểu, phản ánh đúng nghiệp vụ của service.

API Gateway đóng vai trò là điểm truy cập chung. Client gửi request đến Gateway, sau đó Gateway chuyển tiếp request đến User Service, Merchant Service, Order Service hoặc các service liên quan. Việc sử dụng Gateway giúp frontend không cần cấu hình nhiều địa chỉ service khác nhau và giúp hệ thống tập trung kiểm soát xác thực.

4.2. Xác thực và phân quyền

Cơ chế xác thực của hệ thống dựa trên JWT Token. Khi đăng nhập thành công, User Service sinh token chứa thông tin định danh và vai trò người dùng. Token được frontend gửi trong header Authorization của các request cần xác thực. Gateway hoặc service nhận request sẽ kiểm tra token trước khi xử lý nghiệp vụ.

Trong phạm vi đồ án, phân quyền được triển khai ở mức cơ bản nhằm phân biệt người dùng hợp lệ và các thao tác cần đăng nhập. Mô hình này có thể mở rộng trong tương lai để kiểm soát chi tiết hơn theo vai trò customer, merchant, driver hoặc admin.

4.3. Nhóm API chính

Bảng 4.1. Nhóm API chính của hệ thống

Service	Endpoint	Mô tả
User Service	POST /auth/login	Đăng nhập và nhận JWT Token
User Service	GET /users/drivers/available	Tìm tài xế khả dụng cho giao hàng
Merchant Service	GET /merchants	Lấy danh sách nhà hàng
Merchant Service	GET /merchants/{id}/items	Lấy danh sách món ăn của nhà hàng
Merchant Service	POST /items/validate	Kiểm tra giá và tính khả dụng của món
Order Service	POST /orders	Tạo đơn hàng mới

Order Service	PATCH /orders/{id}/status	Cập nhật trạng thái đơn hàng
Delivery Service	GET /deliveries/{orderId}	Lấy thông tin giao hàng theo đơn
Notification Service	POST /logs	Gửi log tập trung từ service khác
Notification Service	WebSocket /socket.io	Cập nhật thời gian thực cho frontend

4.4. Luồng API tiêu biểu

Luồng đăng nhập bắt đầu khi người dùng gửi email và mật khẩu đến API Gateway. Gateway chuyển request đến User Service. User Service kiểm tra thông tin tài khoản, so sánh mật khẩu đã mã hóa và trả về JWT Token nếu xác thực thành công. Frontend lưu token này để sử dụng trong các request tiếp theo.

Luồng đặt hàng phức tạp hơn vì có sự phối hợp giữa nhiều service. Frontend gửi request tạo đơn đến Gateway, Gateway chuyển đến Order Service. Order Service gọi Merchant Service để xác thực danh sách món ăn. Nếu dữ liệu hợp lệ, Order Service lưu đơn hàng và phát sự kiện order.created vào RabbitMQ. Sau đó, Delivery Service và Notification Service tiếp tục xử lý theo cơ chế bất đồng bộ.

4.5. Chuẩn phản hồi và xử lý lỗi

Các API nên trả về phản hồi có cấu trúc thống nhất để frontend dễ xử lý. Một phản hồi thành công thường bao gồm trạng thái xử lý, dữ liệu trả về và thông điệp mô tả. Trong trường hợp lỗi, phản hồi cần chỉ rõ nguyên nhân như thiếu token, dữ liệu không hợp lệ, không tìm thấy tài nguyên hoặc lỗi service nội bộ.

Việc chuẩn hóa phản hồi có ý nghĩa quan trọng trong kiến trúc Microservices. Khi nhiều service cùng cung cấp API, một định dạng phản hồi thống nhất giúp giảm chi phí xử lý ở phía frontend và giúp quá trình debug thuận tiện hơn. Gateway cũng có thể đóng vai trò chuẩn hóa một số lỗi phát sinh khi định tuyến request.

CHƯƠNG 5. LUỒNG MESSAGE QUEUE

5.1. Vai trò của RabbitMQ

RabbitMQ là thành phần trung gian hỗ trợ giao tiếp bất đồng bộ giữa các service trong BiteSwift. Thay vì để Order Service gọi trực tiếp Delivery Service và Notification Service sau khi tạo đơn, Order Service chỉ cần phát một sự kiện vào RabbitMQ. Các service quan tâm đến sự kiện này sẽ tự tiêu thụ và xử lý theo trách nhiệm riêng.

Cách tổ chức dựa trên message queue giúp giảm sự phụ thuộc trực tiếp giữa các service. Order Service không cần biết chính xác Delivery Service hoặc Notification Service đang hoạt động như thế nào. Khi hệ thống mở rộng, có thể bổ sung thêm consumer mới để xử lý cùng một sự kiện mà không cần thay đổi logic chính của Order Service.

5.2. Các sự kiện chính trong hệ thống

Bảng 5.1. Các sự kiện chính trên RabbitMQ

Sự kiện	Nguồn phát	Thành phần xử lý	Ý nghĩa
order.created	Order Service	Delivery Service, Notification Service	Phát sinh sau khi đơn hàng được tạo thành công
delivery.status_changed	Delivery Service	Notification Service, Order Service	Phát sinh khi trạng thái giao hàng thay đổi
service.log	Các service	Notification Service	Gửi log tập trung để hiển thị trên dashboard

5.3. Luồng tạo đơn và phát sự kiện

Khi người dùng xác nhận đặt món, Order Service thực hiện các bước xử lý bắt buộc như kiểm tra món ăn, tính tổng giá trị đơn hàng và lưu đơn hàng vào cơ sở dữ liệu. Sau khi các bước này hoàn tất, service phát sự kiện order.created lên RabbitMQ. Sự kiện này chứa các thông tin cần thiết như mã đơn hàng, người dùng, nhà hàng, danh sách món và địa chỉ giao hàng.

Delivery Service lắng nghe sự kiện order.created để bắt đầu quy trình giao hàng. Notification Service cũng lắng nghe sự kiện này để thông báo cho dashboard hoặc giao diện người dùng rằng đơn hàng mới đã được tạo. Như vậy, cùng một sự kiện có thể kích hoạt

5.4. Luồng cập nhật trạng thái giao hàng

Sau khi nhận đơn hàng mới, Delivery Service mô phỏng các giai đoạn giao hàng bằng cách thay đổi trạng thái theo thời gian. Mỗi khi trạng thái thay đổi, service phát sự kiện `delivery.status_changed`. Notification Service nhận sự kiện này và phát cập nhật thời gian thực đến frontend thông qua WebSocket.

Bên cạnh việc phát event, Delivery Service còn cập nhật ngược trạng thái đơn hàng sang Order Service thông qua REST API. Đây là ví dụ về việc kết hợp hai cơ chế giao tiếp trong cùng một luồng nghiệp vụ: message queue dùng để thông báo sự kiện, còn REST API dùng để cập nhật trạng thái chính thức trong service sở hữu dữ liệu đơn hàng.

5.5. Xử lý lỗi và tính chịu lỗi

Trong hệ thống phân tán, lỗi có thể xuất hiện ở nhiều điểm như service bị dừng, kết nối mạng lỗi hoặc consumer xử lý thất bại. Việc sử dụng RabbitMQ giúp hệ thống giảm rủi ro mất sự kiện trong các tình huống xử lý bất đồng bộ. Message có thể được giữ trong queue cho đến khi consumer sẵn sàng xử lý.

Trong phạm vi đồ án, hệ thống tập trung mô phỏng luồng message cơ bản. Tuy nhiên, về mặt thiết kế, có thể mở rộng thêm cơ chế retry, dead letter queue và idempotency để xử lý các lỗi phức tạp hơn. Idempotency đặc biệt quan trọng nhằm tránh việc một sự kiện bị xử lý lặp lại gây cập nhật sai dữ liệu.

5.6. Ý nghĩa của Message Queue trong đề tài

Message Queue là thành phần thể hiện rõ đặc trưng của hệ thống phân tán trong dự án BiteSwift. Nhờ RabbitMQ, các service không cần phụ thuộc cứng vào nhau trong mọi thao tác. Order Service có thể phản hồi cho người dùng sau khi đơn hàng được tạo, trong khi Delivery Service và Notification Service xử lý phần việc của mình ở phía sau.

Cách tiếp cận này phù hợp với các hệ thống thực tế có nhiều tác vụ nền. Trong bài toán đặt đồ ăn, việc gửi thông báo, tìm tài xế và cập nhật trạng thái giao hàng không nhất thiết phải chặn thao tác tạo đơn. Việc tách các bước này thành các sự kiện giúp hệ thống linh hoạt hơn và dễ mở rộng hơn.

CHƯƠNG 6. THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1. Mục tiêu thử nghiệm

Quá trình thử nghiệm được thực hiện nhằm đánh giá khả năng hoạt động của các chức năng chính trong BiteSwift, đồng thời kiểm chứng sự phối hợp giữa các service trong kiến trúc Microservices. Trọng tâm thử nghiệm bao gồm luồng đăng nhập, tra cứu dữ liệu, tạo đơn hàng, phát message qua RabbitMQ, xử lý giao hàng và cập nhật thời gian thực trên dashboard.

Ngoài kiểm thử chức năng, nhóm cũng quan sát khả năng vận hành của hệ thống khi nhiều thành phần hoạt động đồng thời. Dashboard giám sát được sử dụng như công cụ hỗ trợ trực quan, giúp theo dõi health check, log tập trung và luồng message trong quá trình thực thi.

6.2. Môi trường thử nghiệm

Bảng 6.1. Môi trường thử nghiệm

Thành phần	Mô tả
Hệ điều hành	Windows / Linux / macOS có hỗ trợ Docker
Runtime	Node.js >= 18
Cơ sở dữ liệu	PostgreSQL, MongoDB
Message Broker	RabbitMQ
Frontend	React + Vite chạy trên trình duyệt
Công cụ triển khai	Docker, Docker Compose, npm scripts
Kiểm thử nhanh	npm run test:smoke

6.3. Kịch bản kiểm thử

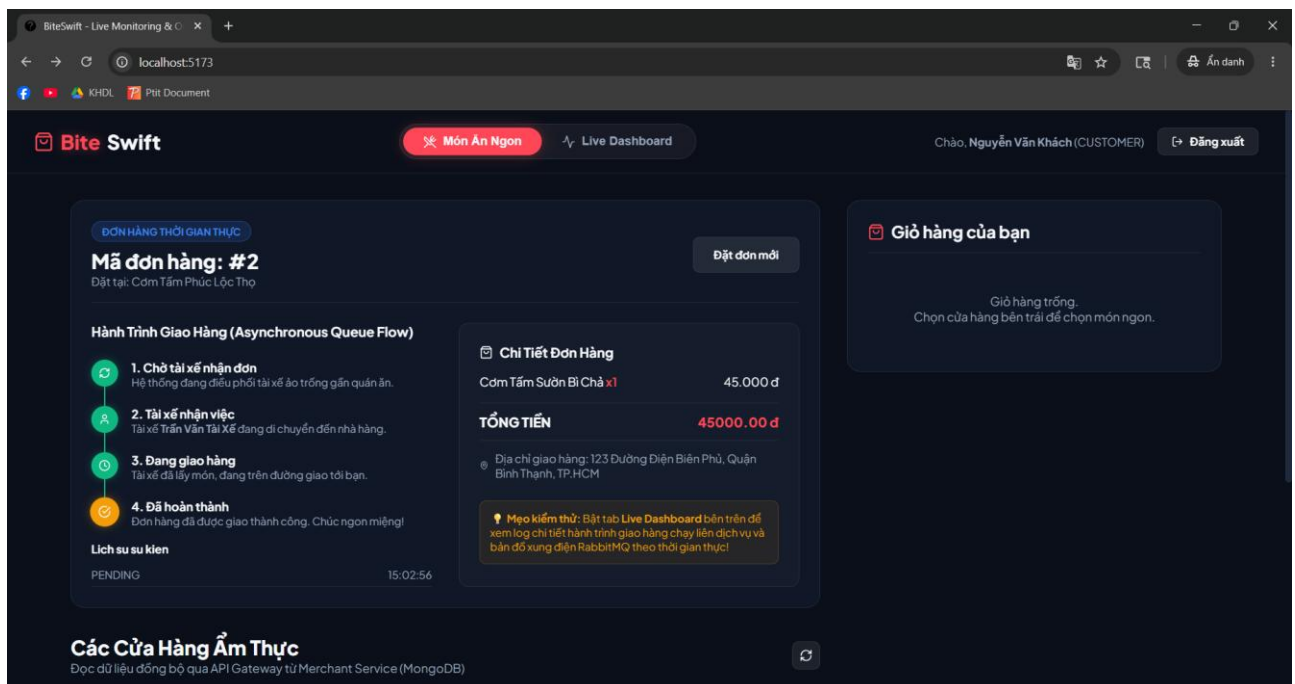
Bảng 6.2. Kịch bản kiểm thử hệ thống

Mã	Kịch bản	Kết quả mong đợi
TC01	Đăng nhập tài khoản mẫu	Người dùng nhận JWT Token và truy cập được hệ thống

TC02	Xem danh sách nhà hàng và món ăn	Frontend hiển thị dữ liệu từ Merchant Service
TC03	Tạo đơn hàng mới	Order Service kiểm tra món ăn, lưu đơn hàng và phát event
TC04	Xử lý message order.created	Delivery Service nhận event và tạo tiến trình giao hàng
TC05	Cập nhật trạng thái giao hàng	Trạng thái đơn hàng thay đổi theo các giai đoạn
TC06	Thông báo thời gian thực	Notification Service phát cập nhật qua WebSocket
TC07	Giám sát dashboard	Dashboard hiển thị health check, log và luồng message

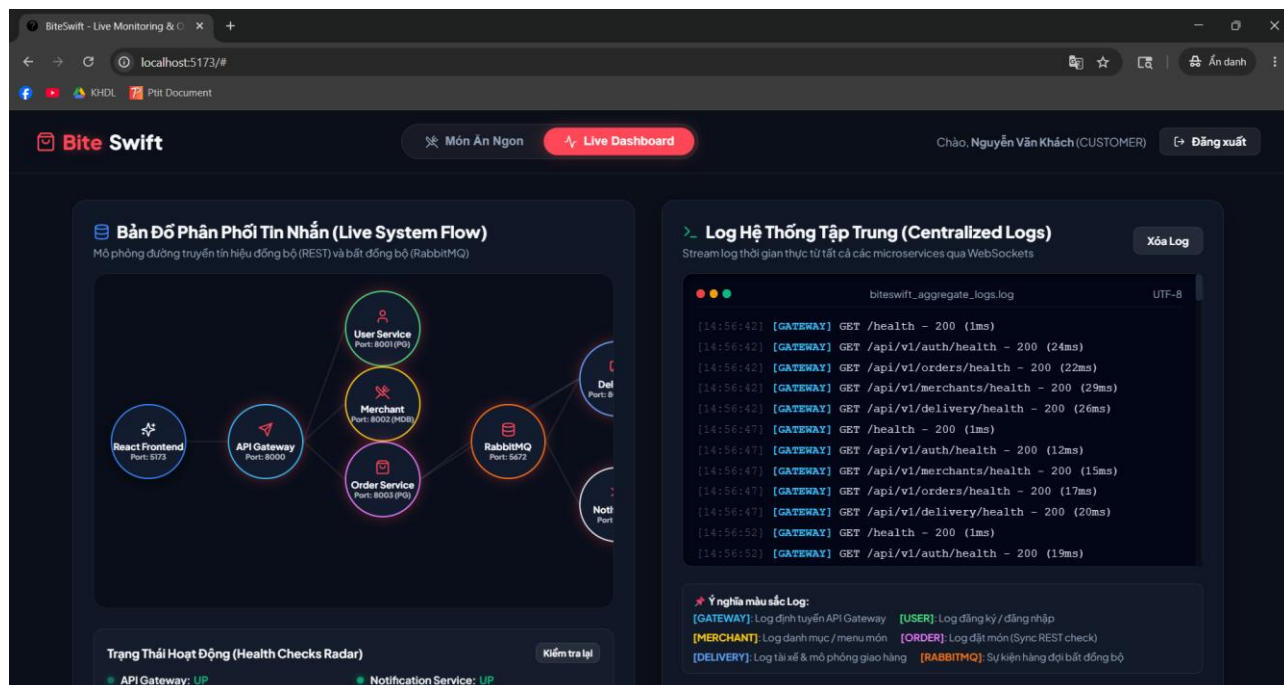
6.4. Kết quả thử nghiệm

Kết quả thử nghiệm cho thấy các service có thể khởi chạy độc lập và phối hợp với nhau thông qua API Gateway, RabbitMQ và WebSocket. Khi thực hiện thao tác đặt món, hệ thống tạo đơn hàng thành công, phát sự kiện order.created và kích hoạt xử lý giao hàng ở Delivery Service.



Hình 2. Kết quả đặt món và cập nhật trạng thái giao hàng theo thời gian thực

Dashboard hiển thị được luồng hoạt động của hệ thống theo thời gian thực. Các log phát sinh từ Gateway, Order Service, Delivery Service và Notification Service được gom tập trung, giúp người quan sát nhận biết trình tự xử lý của từng service. Luồng message từ Order Service đến RabbitMQ và từ RabbitMQ đến các consumer cũng được thể hiện rõ trong quá trình demo.



Hình 3. Live Dashboard hiển thị luồng hệ thống, log tập trung và trạng thái hoạt động của các microservice

Kịch bản smoke test giúp kiểm tra nhanh các thành phần chính của hệ thống. Script kiểm thử gọi qua API Gateway, kiểm tra health, thực hiện một số thao tác CRUD mẫu và tạo đơn hàng để kích hoạt luồng liên service. Đây là cách kiểm thử phù hợp với đồ án vì có thể nhanh chóng chứng minh hệ thống hoạt động xuyên suốt.

6.5. Đánh giá ưu điểm

Ưu điểm nổi bật của hệ thống là thể hiện rõ kiến trúc Microservices. Các service được tách riêng theo miền nghiệp vụ, có cổng chạy độc lập và sử dụng cơ sở dữ liệu phù hợp. Hệ thống không chỉ dừng ở việc chia service mà còn có giao tiếp đồng bộ, giao tiếp bất đồng bộ và cập nhật thời gian thực.

Dashboard giám sát là điểm mạnh quan trọng. Trong nhiều đồ án Microservices, luồng xử lý giữa các service khó quan sát nếu chỉ nhìn vào terminal. BiteSwift cung cấp giao diện trực quan để theo dõi health check, log và message flow, giúp việc trình bày trước giảng viên dễ hiểu và thuyết phục hơn.

Việc sử dụng Docker Compose cho hạ tầng cũng giúp quá trình cài đặt thuận tiện.

PostgreSQL, MongoDB và RabbitMQ có thể được khởi chạy bằng một lệnh, giảm sai khác môi trường giữa các máy. Điều này phù hợp với yêu cầu phát triển và kiểm thử trong phạm vi môn học.

6.6. Hạn chế và hướng phát triển

Hệ thống hiện tại vẫn còn một số hạn chế. Nghiệp vụ giao hàng mới được mô phỏng, chưa tích hợp bản đồ, GPS hoặc thuật toán điều phối tài xế thực tế. Chức năng thanh toán trực tuyến chưa được triển khai. Một số cơ chế nâng cao trong hệ thống phân tán như retry policy, circuit breaker, dead letter queue và idempotency chưa được hiện thực đầy đủ.

Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng. Thứ nhất, bổ sung Payment Service để xử lý thanh toán trực tuyến. Thứ hai, tích hợp bản đồ và vị trí tài xế nhằm mô phỏng giao hàng thực tế hơn. Thứ ba, triển khai Kafka hoặc mở rộng RabbitMQ với dead letter queue để tăng độ tin cậy. Thứ tư, triển khai hệ thống trên Kubernetes nhằm đánh giá khả năng scale và vận hành trong môi trường cloud.

CHƯƠNG 7. HƯỚNG DẪN CÀI ĐẶT

7.1. Yêu cầu môi trường

Để cài đặt và chạy hệ thống BiteSwift, máy tính cần có Node.js phiên bản 18 trở lên, Docker và Docker Compose. Node.js được sử dụng để chạy các service backend và frontend. Docker Compose được sử dụng để khởi chạy các thành phần hạ tầng như PostgreSQL, MongoDB và RabbitMQ.

Bảng 7.1. Yêu cầu môi trường triển khai

Công cụ	Phiên bản / Yêu cầu	Mục đích
Node.js	$\geq 18.0.0$	Chạy backend service và frontend
Docker	Bản ổn định mới	Chạy container hạ tầng
Docker Compose	Tích hợp trong Docker Desktop hoặc CLI	Khởi chạy PostgreSQL, MongoDB, RabbitMQ
Trình duyệt web	Chrome, Edge hoặc Firefox	Truy cập frontend và dashboard
Terminal	PowerShell, CMD hoặc Bash	Thực thi lệnh cài đặt và khởi động

7.2. Tải mã nguồn

Mã nguồn dự án được lưu trữ trên GitHub. Người dùng có thể tải mã nguồn bằng cách clone repository hoặc tải file nén trực tiếp từ trang GitHub. Sau khi tải về, mở terminal tại thư mục gốc của dự án để thực hiện các bước cài đặt tiếp theo.

```
git clone https://github.com/phuc19416/TEST.git
```

```
cd TEST
```

7.3. Khởi chạy hạ tầng Docker

Trước khi chạy các service, cần khởi động các thành phần hạ tầng gồm PostgreSQL, MongoDB và RabbitMQ.

Trong lần chạy đầu tiên, hệ thống sẽ tạo các cơ sở dữ liệu cần thiết cho User Service, Order Service và Delivery Service. RabbitMQ Management UI có thể được truy cập để quan sát exchange, queue và message.

7.4. Cài đặt dependencies

Sau khi hạ tầng Docker đã chạy, cần cài đặt thư viện cho toàn bộ các workspace của

dự án.

Cài đặt dependencies cho frontend và các service backend trong monorepo. Việc quản lý dependencies theo workspace giúp đơn giản hóa quá trình cài đặt, đặc biệt khi dự án có nhiều service Node.js độc lập.

7.5. Khởi chạy toàn bộ hệ thống

Khi hạ tầng và dependencies đã sẵn sàng, có thể khởi chạy toàn bộ backend service và frontend

Sau khi khởi chạy, frontend, API Gateway và các service sẽ hoạt động trên các cổng tương ứng. Người dùng truy cập frontend để thực hiện thao tác đặt món và dashboard để quan sát hoạt động hệ thống.

Bảng 7.2. Cổng truy cập và thành phần tương ứng

Cổng	Thành phần	Mục đích
5173	Frontend & Dashboard	Giao diện đặt món và giám sát
8000	API Gateway	Cổng vào REST API
8001	User Service	Quản lý người dùng
8002	Merchant Service	Quản lý nhà hàng và món ăn
8003	Order Service	Xử lý đơn hàng
8004	Delivery Service	Xử lý giao hàng
8005	Notification Service	WebSocket, log và thông báo
15672	RabbitMQ Management UI	Quản trị message broker

7.6. Kiểm thử nhanh hệ thống

Sau khi hệ thống đã chạy, có thể sử dụng smoke test để kiểm tra nhanh các chức năng chính. Kiểm thử gọi qua API Gateway, kiểm tra health của service, thực hiện một số thao tác dữ liệu mẫu và tạo đơn hàng để kích hoạt luồng RabbitMQ.

Nếu smoke test hoàn tất thành công, có thể xem đây là dấu hiệu hệ thống đã được cài đặt đúng. Ngoài ra, người dùng nên mở dashboard để kiểm tra log tập trung và luồng message khi tạo đơn hàng từ giao diện.

7.7. Chạy toàn bộ bằng Docker Compose

Ngoài cách chạy bằng npm script, dự án còn hỗ trợ build và chạy toàn bộ frontend cùng các microservice trong container. Cách này phù hợp khi muốn kiểm thử hệ thống gần với môi trường triển khai thực tế hơn.

7.8. Một số lỗi thường gặp

Trong quá trình cài đặt, lỗi thường gặp nhất là Docker chưa chạy hoặc cổng đã bị ứng dụng khác chiếm dụng. Khi gặp lỗi kết nối tới PostgreSQL, MongoDB hoặc RabbitMQ, cần kiểm tra trạng thái container bằng lệnh docker ps. Nếu frontend không kết nối được backend, cần kiểm tra API Gateway có chạy trên cổng 8000 hay không.

Trường hợp cài đặt dependencies thất bại, cần kiểm tra phiên bản Node.js và xóa thư mục node_modules trước khi cài đặt lại. Với RabbitMQ, có thể truy cập giao diện quản trị tại cổng 15672 để xác nhận broker đã hoạt động. Các lỗi liên quan tới message thường có thể được phát hiện thông qua log ở terminal hoặc dashboard.

KẾT LUẬN

Báo cáo đã trình bày quá trình phân tích và thiết kế hệ thống BiteSwift - một ứng dụng đặt đồ ăn trực tuyến được xây dựng theo kiến trúc Microservices. Hệ thống được chia thành nhiều service độc lập, bao gồm API Gateway, User Service, Merchant Service, Order Service, Delivery Service và Notification Service. Mỗi service đảm nhiệm một miền nghiệp vụ riêng và phối hợp với nhau thông qua RESTful API, RabbitMQ và WebSocket.

Về mặt kỹ thuật, dự án đáp ứng được các yêu cầu cơ bản của chủ đề phát triển ứng dụng Microservice. RESTful API được sử dụng cho giao tiếp đồng bộ giữa frontend, Gateway và các service. RabbitMQ được sử dụng cho giao tiếp bất đồng bộ giữa Order Service, Delivery Service và Notification Service. Bên cạnh đó, hệ thống sử dụng cơ sở dữ liệu riêng cho từng service và cung cấp dashboard giám sát thời gian thực.

Thông qua quá trình thực hiện, nhóm đã vận dụng được các kiến thức về hệ thống phân tán, tách dịch vụ, giao tiếp liên dịch vụ, message queue, xác thực JWT và triển khai hạ tầng bằng Docker. Mặc dù hệ thống còn một số hạn chế như chưa tích hợp thanh toán thực, GPS và các cơ chế chịu lỗi nâng cao, BiteSwift đã thể hiện được luồng nghiệp vụ chính và các đặc trưng quan trọng của kiến trúc Microservices.

Trong tương lai, hệ thống có thể được tiếp tục phát triển theo hướng hoàn thiện nghiệp vụ thực tế, bổ sung Payment Service, tích hợp bản đồ, cải thiện bảo mật, triển khai dead letter queue, bổ sung quan sát bằng Prometheus/Grafana và triển khai lên Kubernetes. Những hướng phát triển này sẽ giúp BiteSwift tiến gần hơn đến một hệ thống phân tán hoàn chỉnh trong môi trường sản xuất.

TÀI LIỆU THAM KHẢO

1. Repository dự án BiteSwift:
<https://github.com/phuc19416/HTPTN11>
2. Tài liệu Node.js: <https://nodejs.org/>
3. Tài liệu Docker: <https://docs.docker.com/>
4. Tài liệu RabbitMQ: <https://www.rabbitmq.com/documentation.html>
5. Tài liệu MongoDB: <https://www.mongodb.com/docs/>
6. Tài liệu PostgreSQL: <https://www.postgresql.org/docs/>
7. Tài liệu Socket.io: <https://socket.io/docs/>
8. Sam Newman, Building Microservices: Designing Fine-Grained Systems, O'Reilly Media.